

```
In [1]: import os
import shutil

SOURCE_DIR = "/kaggle/input/datasets/sprakashbaranwal/saved-pre-trained-model-zip/s
DEST_DIR = "/kaggle/working/saved_pre_trained_model"

# Remove existing destination if present
if os.path.exists(DEST_DIR):
    shutil.rmtree(DEST_DIR)

# Copy the pre-trained model to /kaggle/working
shutil.copytree(SOURCE_DIR, DEST_DIR)

print("=" * 70)
print("[Model Setup] Pre-trained model copied successfully")
print(f"[Source]      {SOURCE_DIR}")
print(f"[Destination] {DEST_DIR}")
print("=" * 70)

# Verify copied files
for root, dirs, files in os.walk(DEST_DIR):
    for file in files:
        print(os.path.join(root, file))

=====
[Model Setup] Pre-trained model copied successfully
[Source]      /kaggle/input/datasets/sprakashbaranwal/saved-pre-trained-model-zip/sa
ved_pre_trained_model
[Destination] /kaggle/working/saved_pre_trained_model
=====
/kaggle/working/saved_pre_trained_model/generation_config.json
/kaggle/working/saved_pre_trained_model/tokenizer_config.json
/kaggle/working/saved_pre_trained_model/config.json
/kaggle/working/saved_pre_trained_model/model.safetensors
/kaggle/working/saved_pre_trained_model/tokenizer.json
```

```
In [2]: """
=====
File Name : train.py

Purpose:
    Fine-tuning pipeline for T5-small Encoder-Decoder model on SQuAD v1.1.

    Final experiment configuration:
        - Dataset samples : 20,000
        - Train split      : 80% = 16,000
        - Validation       : 10% = 2,000
        - Test             : 10% = 2,000
        - Epochs           : 3
        - Optimizer        : AdamW
        - Learning rate    : 5e-5
        - Warmup           : 10% of total training steps
        - Batch size       : 16 on CUDA
        - Input length     : 512 tokens
        - Target length    : 64 tokens
```

- FP16 : Enabled on CUDA

Generated artifacts:

- saved_model/
- training_log.csv
- loss_curve.png
- test_results.json

Course : Natural Language Processing (S2-25_AIMLCZG530)

Program : M.Tech. in AML, BITS Pilani (WILP)

Group : Group 238

=====

=====

1. IMPORTS

=====

```
import sys
import os
import time
import json
from pathlib import Path

import torch
import pandas as pd

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from datasets import load_dataset, DatasetDict
```

```
from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM,
    Trainer,
    TrainingArguments,
    TrainerCallback
)
```

=====

2. REPRODUCIBILITY

=====

SEED = 42

```
torch.manual_seed(SEED)
```

```
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

=====

3. PATH DEFINITIONS

```

# =====

# Works when executed as:
#     python train.py
#
# Also works when executed inside Jupyter/Kaggle:
#     %run train.py

try:
    BASE_DIR = Path(__file__).resolve().parent
except NameError:
    BASE_DIR = Path.cwd()

# Local pre-trained T5 model uploaded to Kaggle
BASE_MODEL = "/kaggle/working/saved_pre_trained_model"

# Output artifacts
SAVED_MODEL_DIR = BASE_DIR / "saved_model"
CSV_LOG_PATH = BASE_DIR / "training_log.csv"
PLOT_IMAGE_PATH = BASE_DIR / "loss_curve.png"
TEST_METRICS_PATH = BASE_DIR / "test_results.json"

# =====
# 4. FINAL EXPERIMENT CONFIGURATION
# =====

DEFAULT_NUM_SAMPLES = 20000
DEFAULT_BATCH_SIZE = 16

NUM_EPOCHS = 3

LEARNING_RATE = 5e-5

WARMUP_RATIO = 0.10

MAX_INPUT_LENGTH = 512
MAX_TARGET_LENGTH = 64

LOGGING_STEPS = 50
EVAL_STEPS = 50

# =====
# 5. DATASET PREPARATION
# =====

def prepare_squad_data(tokenizer, num_samples=DEFAULT_NUM_SAMPLES):
    """
    Load SQuAD v1.1, highlight the target answer span using <hl> tags,
    tokenize the input and target question, and create an 80/10/10 split.

    For 20,000 samples:
        Train      = 16,000
        Validation = 2,000
    """

```

```

Test      = 2,000
"""

print(
    f"[1/4] Loading SQuAD v1.1 "
    f"(sample subset size: {num_samples})...",
    flush=True
)

raw_dataset = load_dataset(
    "rajpurkar/squad",
    split=f"train[:{num_samples}]"
)

# -----
# Preprocessing
# -----

def preprocess_function(examples):

    inputs = []
    targets = []

    for context, answer, question in zip(
        examples["context"],
        examples["answers"],
        examples["question"]
    ):

        # SQuAD answer text
        ans_text = answer["text"][0]

        # Highlight the first occurrence of the answer
        if ans_text in context:

            highlighted_context = context.replace(
                ans_text,
                f"<h1> {ans_text} <h1>",
                1
            )

            input_text = (
                f"generate question: context: "
                f"{highlighted_context}"
            )

        else:

            # Fallback if answer text is not found
            input_text = (
                f"generate question: context: {context}"
            )

        inputs.append(input_text)
        targets.append(question)

```

```

# -----
# Tokenize input/context
# -----

model_inputs = tokenizer(
    inputs,
    max_length=MAX_INPUT_LENGTH,
    truncation=True,
    padding="max_length"
)

# -----
# Tokenize target questions
# -----

labels = tokenizer(
    targets,
    max_length=MAX_TARGET_LENGTH,
    truncation=True,
    padding="max_length"
)

# -----
# Ignore padding tokens during loss calculation
# -----

labels_ids = [
    [
        token if token != tokenizer.pad_token_id else -100
        for token in label
    ]
    for label in labels["input_ids"]
]

model_inputs["labels"] = labels_ids

return model_inputs

# -----
# Apply preprocessing
# -----

processed_dataset = raw_dataset.map(
    preprocess_function,
    batched=True,
    remove_columns=raw_dataset.column_names
)

# -----
# 80% Train / 20% Temporary
# -----

train_test_split = processed_dataset.train_test_split(
    test_size=0.20,
    seed=SEED
)

```

```

# -----
# Split temporary 20% equally:
# 10% Validation / 10% Test
# -----

val_test_split = train_test_split["test"].train_test_split(
    test_size=0.50,
    seed=SEED
)

split_dataset = DatasetDict({
    "train": train_test_split["train"],
    "validation": val_test_split["train"],
    "test": val_test_split["test"]
})

print(
    f"[Dataset Split] "
    f"Train: {len(split_dataset['train'])} | "
    f"Validation: {len(split_dataset['validation'])} | "
    f"Test: {len(split_dataset['test'])}",
    flush=True
)

return split_dataset

# =====
# 6. TRAINING / VALIDATION LOSS LOGGER
# =====

class TrainingLoggerCallback(TrainerCallback):
    """
    Captures training loss and validation loss during training.

    Results are saved to:
        training_log.csv

    A combined training/validation loss graph is saved to:
        loss_curve.png
    """

    def __init__(self):

        self.train_history = []
        self.eval_history = []

    def on_log(
        self,
        args,
        state,
        control,
        logs=None,
        **kwargs
    ):

```

```

if not logs:
    return

epoch_val = (
    round(state.epoch, 2)
    if state.epoch is not None
    else 0.0
)

# -----
# Training Loss
# -----

if "loss" in logs:

    loss_val = round(float(logs["loss"]), 4)

    self.train_history.append({
        "step": state.global_step,
        "epoch": epoch_val,
        "train_loss": loss_val
    })

    print(
        f"[Training Step {state.global_step}] "
        f"Epoch {epoch_val} | "
        f"Training Loss: {loss_val}",
        flush=True
    )

# -----
# Validation Loss
# -----

if "eval_loss" in logs:

    eval_loss_val = round(
        float(logs["eval_loss"]),
        4
    )

    self.eval_history.append({
        "step": state.global_step,
        "epoch": epoch_val,
        "eval_loss": eval_loss_val
    })

    print(
        f"[Validation Step {state.global_step}] "
        f"Epoch {epoch_val} | "
        f"Validation Loss: {eval_loss_val}",
        flush=True
    )

def export_artifacts(self):

```

```

df_train = pd.DataFrame(self.train_history)
df_eval = pd.DataFrame(self.eval_history)

# -----
# Create combined CSV
# -----

if not df_train.empty and not df_eval.empty:

    df_combined = pd.merge(
        df_train,
        df_eval,
        on=["step", "epoch"],
        how="outer"
    ).sort_values("step")

elif not df_train.empty:

    df_combined = df_train

else:

    df_combined = df_eval

df_combined.to_csv(
    str(CSV_LOG_PATH),
    index=False
)

print(
    f"[Artifact] Unified training/validation metrics "
    f"log saved to '{CSV_LOG_PATH.name}'",
    flush=True
)

# -----
# Generate loss curve
# -----

plt.figure(figsize=(9, 5))

if not df_train.empty:

    plt.plot(
        df_train["step"],
        df_train["train_loss"],
        marker="o",
        linewidth=2,
        label="Training Loss"
    )

if not df_eval.empty:

    plt.plot(
        df_eval["step"],

```



```

        df_eval["eval_loss"],
        marker="s",
        linewidth=2,
        linestyle="--",
        label="Validation Loss"
    )

plt.title(
    "Step-Level Training and Validation Loss "
    "(T5-Small on SQuAD v1.1)"
)

plt.xlabel(
    "Optimization Step "
    "(Logged Every 50 Steps)"
)

plt.ylabel("Cross-Entropy Loss")

plt.grid(
    True,
    linestyle="--",
    alpha=0.6
)

plt.legend()

plt.tight_layout()

plt.savefig(
    str(PLOT_IMAGE_PATH),
    dpi=300
)

plt.close()

print(
    f"[Artifact] Step-level loss graph exported to "
    f"'{PLOT_IMAGE_PATH.name}'",
    flush=True
)

# =====
# 7. DEVICE DETECTION
# =====

def detect_device():
    """
    Detect CUDA GPU, Apple MPS GPU, or CPU.

    CUDA:
        Batch size = 16
        FP16 = True

    MPS / CPU:

```

```

        Batch size = 8
        FP16 = False
    """

    if torch.cuda.is_available():

        device_name = torch.cuda.get_device_name(0)
        gpu_count = torch.cuda.device_count()

        return (
            "cuda",
            f"CUDA GPU ({device_name}) x{gpu_count}",
            True,
            16
        )

    elif (
        hasattr(torch.backends, "mps")
        and torch.backends.mps.is_available()
    ):

        return (
            "mps",
            "Apple Silicon MPS (GPU)",
            False,
            8
        )

    else:

        return (
            "cpu",
            "System CPU",
            False,
            8
        )

# =====
# 8. MAIN TRAINING PIPELINE
# =====

def main():

    start_time = time.time()

    # -----
    # Runtime configuration
    # -----

    num_samples = int(
        os.environ.get(
            "NUM_SAMPLES",
            str(DEFAULT_NUM_SAMPLES)
        )
    )

```

```

requested_batch_size = int(
    os.environ.get(
        "BATCH_SIZE",
        "0"
    )
)

# -----
# Device
# -----

(
    device_type,
    device_label,
    fp16_enabled,
    detected_batch_size
) = detect_device()

batch_size = (
    requested_batch_size
    if requested_batch_size > 0
    else detected_batch_size
)

# -----
# System information
# -----

print("=" * 78, flush=True)

print(
    "[System Init] Automatic device detection",
    flush=True
)

print(
    f"[System Init] Device      : {device_label}",
    flush=True
)

print(
    f"[System Init] CUDA        : "
    f"{torch.cuda.is_available()}",
    flush=True
)

print(
    f"[System Init] GPU count   : "
    f"{torch.cuda.device_count() if torch.cuda.is_available() else 0}",
    flush=True
)

print(
    f"[System Init] Batch size  : {batch_size}",
    flush=True
)

```

```

)

print(
    f"[System Init] FP16          : {fp16_enabled}",
    flush=True
)

print(
    f"[System Init] Samples      : {num_samples}",
    flush=True
)

print(
    f"[System Init] Epochs       : {NUM_EPOCHS}",
    flush=True
)

print("=" * 78, flush=True)

# -----
# GPU information
# -----

if torch.cuda.is_available():

    for i in range(torch.cuda.device_count()):

        props = torch.cuda.get_device_properties(i)

        total_gb = (
            props.total_memory
            / (1024 ** 3)
        )

        print(
            f"[GPU {i}] {props.name} | "
            f"VRAM: {total_gb:.2f} GB",
            flush=True
        )

# =====
# MODEL LOADING
# =====

phase_start = time.time()

print(
    "\n[Model] Loading local pre-trained model...",
    flush=True
)

print(
    f"[Model] Path: {BASE_MODEL}",
    flush=True
)

```

```

tokenizer = AutoTokenizer.from_pretrained(
    BASE_MODEL,
    local_files_only=True
)

model = AutoModelForSeq2SeqLM.from_pretrained(
    BASE_MODEL,
    local_files_only=True
)

print(
    f"[Timing] Model/tokenizer load: "
    f"{time.time() - phase_start:.1f}s",
    flush=True
)

# =====
# DATASET PREPARATION
# =====

phase_start = time.time()

dataset_split = prepare_squad_data(
    tokenizer,
    num_samples=num_samples
)

print(
    f"[Timing] Dataset preparation: "
    f"{time.time() - phase_start:.1f}s",
    flush=True
)

# =====
# CALLBACK
# =====

logger_callback = TrainingLoggerCallback()

# =====
# TRAINING STEP CALCULATION
# =====

train_samples = len(
    dataset_split["train"]
)

steps_per_epoch = (
    train_samples + batch_size - 1
) // batch_size

total_training_steps = (
    steps_per_epoch * NUM_EPOCHS
)

warmup_steps = max(

```

```

    1,
    int(total_training_steps * WARMUP_RATIO)
)

print(
    "\n[Training Configuration]",
    flush=True
)

print(
    f"Training samples      : {train_samples}",
    flush=True
)

print(
    f"Validation samples      : "
    f"{len(dataset_split['validation'])}",
    flush=True
)

print(
    f"Test samples            : "
    f"{len(dataset_split['test'])}",
    flush=True
)

print(
    f"Epochs                  : {NUM_EPOCHS}",
    flush=True
)

print(
    f"Batch size              : {batch_size}",
    flush=True
)

print(
    f"Steps per epoch         : {steps_per_epoch}",
    flush=True
)

print(
    f"Total training steps    : "
    f"{total_training_steps}",
    flush=True
)

print(
    f"Learning rate           : {LEARNING_RATE}",
    flush=True
)

print(
    f"Warmup ratio            : {WARMUP_RATIO}",
    flush=True
)

```

```

print(
    f"Warmup steps          : {warmup_steps}",
    flush=True
)

print(
    f"Input max length      : "
    f"{MAX_INPUT_LENGTH}",
    flush=True
)

print(
    f"Target max length      : "
    f"{MAX_TARGET_LENGTH}",
    flush=True
)

# =====
# TRAINING ARGUMENTS
# =====

training_args = TrainingArguments(

    output_dir=str(SAVED_MODEL_DIR),

    # -----
    # Training
    # -----

    num_train_epochs=NUM_EPOCHS,

    per_device_train_batch_size=batch_size,

    per_device_eval_batch_size=batch_size,

    learning_rate=LEARNING_RATE,

    optim="adamw_torch",

    # Dynamic 10% warmup
    warmup_steps=warmup_steps,

    # -----
    # Logging / Evaluation
    # -----

    logging_steps=LOGGING_STEPS,

    eval_strategy="steps",

    eval_steps=EVAL_STEPS,

    # -----
    # Saving
    # -----

```

```

save_strategy="no",

# We save manually after training.
# This keeps the output directory clean.
# -----

# GPU
fp16=fp16_enabled,

# -----
# Reporting
# -----

report_to="none",

disable_tqdm=True,

# -----
# Data Loading
# -----

dataloader_pin_memory=torch.cuda.is_available(),

dataloader_num_workers=(
    2 if torch.cuda.is_available() else 0
)
)

# =====
# TRAINER
# =====

trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=dataset_split["train"],

    eval_dataset=dataset_split["validation"],

    callbacks=[logger_callback]
)

# =====
# FINE-TUNING
# =====

print(
    "\n[2/4] Starting model fine-tuning process...",
    flush=True
)

train_start = time.time()

```



```

trainer.train()

# Synchronize CUDA before timing
if torch.cuda.is_available():

    torch.cuda.synchronize()

train_elapsed = (
    time.time() - train_start
)

train_mins, train_secs = divmod(
    int(train_elapsed),
    60
)

print(
    f"[Timing] Fine-tuning duration: "
    f"{train_mins}m {train_secs}s",
    flush=True
)

# =====
# GPU MEMORY
# =====

if torch.cuda.is_available():

    for i in range(torch.cuda.device_count()):

        allocated = (
            torch.cuda.memory_allocated(i)
            / (1024 ** 3)
        )

        reserved = (
            torch.cuda.memory_reserved(i)
            / (1024 ** 3)
        )

        print(
            f"[GPU {i}] Memory - "
            f"Allocated: {allocated:.2f} GB | "
            f"Reserved: {reserved:.2f} GB",
            flush=True
        )

# =====
# HELD-OUT TEST SET
# =====

print(
    "\n[2.5/4] Evaluating fine-tuned model "
    "on held-out Test Set...",
    flush=True
)

```

```

)

test_start = time.time()

test_results = trainer.evaluate(
    eval_dataset=dataset_split["test"],
    metric_key_prefix="test"
)

if torch.cuda.is_available():

    torch.cuda.synchronize()

test_elapsed = (
    time.time() - test_start
)

print(
    f"[Timing] Held-out test evaluation: "
    f"{test_elapsed:.1f}s",
    flush=True
)

test_loss = round(
    float(test_results.get("test_loss", 0.0)),
    4
)

print(
    f"[Test Set Metrics] "
    f"Held-Out Test Loss: {test_loss}",
    flush=True
)

# =====
# SAVE TEST RESULTS
# =====

test_metrics = {

    "test_loss": test_loss,

    "eval_samples": len(
        dataset_split["test"]
    ),

    "num_total_samples": num_samples,

    "train_samples": len(
        dataset_split["train"]
    ),

    "validation_samples": len(
        dataset_split["validation"]
    ),

```

```

        "epochs": NUM_EPOCHS,

        "batch_size": batch_size,

        "learning_rate": LEARNING_RATE,

        "warmup_steps": warmup_steps,

        "max_input_length": MAX_INPUT_LENGTH,

        "max_target_length": MAX_TARGET_LENGTH,

        "model": BASE_MODEL,

        "device": device_label
    }

    with open(
        TEST_METRICS_PATH,
        "w",
        encoding="utf-8"
    ) as f:

        json.dump(
            test_metrics,
            f,
            indent=4
        )

    print(
        f"[Artifact] Held-out test evaluation metrics "
        f"saved to '{TEST_METRICS_PATH.name}'",
        flush=True
    )

    # =====
    # SAVE MODEL
    # =====

    print(
        "\n[3/4] Saving model weights and tokenizer...",
        flush=True
    )

    SAVED_MODEL_DIR.mkdir(
        parents=True,
        exist_ok=True
    )

    model.save_pretrained(
        str(SAVED_MODEL_DIR)
    )

    tokenizer.save_pretrained(
        str(SAVED_MODEL_DIR)
    )

```

```

print(
    f"[System Check] Model weights successfully "
    f"saved to '{SAVED_MODEL_DIR.name}'",
    flush=True
)

# =====
# GENERATE TRAINING ARTIFACTS
# =====

print(
    "\n[4/4] Generating training artifacts...",
    flush=True
)

logger_callback.export_artifacts()

# =====
# FINAL SUMMARY
# =====

elapsed_time = (
    time.time() - start_time
)

mins, secs = divmod(
    int(elapsed_time),
    60
)

print("=" * 78, flush=True)

print(
    f"[Execution Summary] "
    f"Device: {device_label} | "
    f"Samples: {num_samples} | "
    f"Epochs: {NUM_EPOCHS} | "
    f"Total Duration: {mins}m {secs}s | "
    f"Test Loss: {test_loss}",
    flush=True
)

print("-" * 78, flush=True)

print(
    "[Artifacts]",
    flush=True
)

print(
    f"Model : {SAVED_MODEL_DIR}",
    flush=True
)

print(

```

```

        f" Test Results: {TEST_METRICS_PATH}",
        flush=True
    )

    print(
        f" Training Log: {CSV_LOG_PATH}",
        flush=True
    )

    print(
        f" Loss Curve : {PLOT_IMAGE_PATH}",
        flush=True
    )

    print("=" * 78, flush=True)

    print(
        "[Complete] Fine-tuning pipeline executed successfully!",
        flush=True
    )

# =====
# 9. TOP-LEVEL EXCEPTION HANDLER
# =====

if __name__ == "__main__":

    try:

        main()

    except KeyboardInterrupt:

        print(
            "\n[Interrupted] Fine-tuning execution "
            "was stopped by the user.",
            flush=True
        )

    except Exception as e:

        print(
            "\n[ERROR] Fine-tuning pipeline failed: "
            f"{type(e).__name__}: {e}",
            file=sys.stderr,
            flush=True
        )

    # In a normal Python script, return a non-zero exit code.
    # Avoid sys.exit() inside a Jupyter/IPython environment.
    if "IPython" not in sys.modules:

        sys.exit(1)

```

```

=====
[System Init] Automatic device detection
[System Init] Device      : CUDA GPU (Tesla T4) x2
[System Init] CUDA       : True
[System Init] GPU count  : 2
[System Init] Batch size : 16
[System Init] FP16       : True
[System Init] Samples    : 20000
[System Init] Epochs     : 3
=====

[GPU 0] Tesla T4 | VRAM: 14.56 GB
[GPU 1] Tesla T4 | VRAM: 14.56 GB

[Model] Loading local pre-trained model...
[Model] Path: /kaggle/working/saved_pre_trained_model
Loading weights:  0%|          | 0/131 [00:00<?, ?it/s]
[Timing] Model/tokenizer load: 0.7s
[1/4] Loading SQuAD v1.1 (sample subset size: 20000)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md: 0.00B [00:00, ?B/s]
plain_text/train-00000-of-00001.parquet:  0%|          | 0.00/14.5M [00:00<?, ?B/s]
plain_text/validation-00000-of-00001.par(...):  0%|          | 0.00/1.82M [00:00<?, ?B/s]
Generating train split:  0%|          | 0/87599 [00:00<?, ? examples/s]
Generating validation split:  0%|          | 0/10570 [00:00<?, ? examples/s]
Map:  0%|          | 0/20000 [00:00<?, ? examples/s]
[Dataset Split] Train: 16000 | Validation: 2000 | Test: 2000
[Timing] Dataset preparation: 13.4s

[Training Configuration]
Training samples      : 16000
Validation samples    : 2000
Test samples         : 2000
Epochs              : 3
Batch size           : 16
Steps per epoch      : 1000
Total training steps  : 3000
Learning rate        : 5e-05
Warmup ratio         : 0.1
Warmup steps         : 300
Input max length     : 512
Target max length    : 64

[2/4] Starting model fine-tuning process...
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning:
Was asked to gather along dimension 0, but all input tensors were scalars; will instead
unsqueeze and return a vector.
  return super().apply(*args, **kwargs) # type: ignore[misc]

```

```
[Training Step 50] Epoch 0.1 | Training Loss: 3.4865
{'loss': '3.486', 'grad_norm': '3.747', 'learning_rate': '8.167e-06', 'epoch': '0.1'}
[Validation Step 50] Epoch 0.1 | Validation Loss: 3.0804
{'eval_loss': '3.08', 'eval_runtime': '13.56', 'eval_samples_per_second': '147.4', 'eval_steps_per_second': '4.644', 'epoch': '0.1'}
[Training Step 100] Epoch 0.2 | Training Loss: 3.5148
{'loss': '3.515', 'grad_norm': '3.049', 'learning_rate': '1.65e-05', 'epoch': '0.2'}
[Validation Step 100] Epoch 0.2 | Validation Loss: 3.0656
{'eval_loss': '3.066', 'eval_runtime': '14.16', 'eval_samples_per_second': '141.2', 'eval_steps_per_second': '4.449', 'epoch': '0.2'}
[Training Step 150] Epoch 0.3 | Training Loss: 3.5009
{'loss': '3.501', 'grad_norm': '3.727', 'learning_rate': '2.483e-05', 'epoch': '0.3'}
[Validation Step 150] Epoch 0.3 | Validation Loss: 3.0491
{'eval_loss': '3.049', 'eval_runtime': '14.79', 'eval_samples_per_second': '135.2', 'eval_steps_per_second': '4.258', 'epoch': '0.3'}
[Training Step 200] Epoch 0.4 | Training Loss: 3.3651
{'loss': '3.365', 'grad_norm': '3.819', 'learning_rate': '3.317e-05', 'epoch': '0.4'}
[Validation Step 200] Epoch 0.4 | Validation Loss: 3.0508
{'eval_loss': '3.051', 'eval_runtime': '14.39', 'eval_samples_per_second': '139', 'eval_steps_per_second': '4.377', 'epoch': '0.4'}
[Training Step 250] Epoch 0.5 | Training Loss: 3.4368
{'loss': '3.437', 'grad_norm': '3.562', 'learning_rate': '4.15e-05', 'epoch': '0.5'}
[Validation Step 250] Epoch 0.5 | Validation Loss: 3.0518
{'eval_loss': '3.052', 'eval_runtime': '14.65', 'eval_samples_per_second': '136.5', 'eval_steps_per_second': '4.301', 'epoch': '0.5'}
[Training Step 300] Epoch 0.6 | Training Loss: 3.4158
{'loss': '3.416', 'grad_norm': '3.362', 'learning_rate': '4.983e-05', 'epoch': '0.6'}
[Validation Step 300] Epoch 0.6 | Validation Loss: 3.0479
{'eval_loss': '3.048', 'eval_runtime': '14.77', 'eval_samples_per_second': '135.4', 'eval_steps_per_second': '4.265', 'epoch': '0.6'}
[Training Step 350] Epoch 0.7 | Training Loss: 3.443
{'loss': '3.443', 'grad_norm': '3.488', 'learning_rate': '4.796e-05', 'epoch': '0.7'}
[Validation Step 350] Epoch 0.7 | Validation Loss: 3.047
{'eval_loss': '3.047', 'eval_runtime': '15.17', 'eval_samples_per_second': '131.8', 'eval_steps_per_second': '4.152', 'epoch': '0.7'}
[Training Step 400] Epoch 0.8 | Training Loss: 3.5223
{'loss': '3.522', 'grad_norm': '2.95', 'learning_rate': '4.588e-05', 'epoch': '0.8'}
[Validation Step 400] Epoch 0.8 | Validation Loss: 3.0414
{'eval_loss': '3.041', 'eval_runtime': '14.88', 'eval_samples_per_second': '134.4', 'eval_steps_per_second': '4.234', 'epoch': '0.8'}
[Training Step 450] Epoch 0.9 | Training Loss: 3.3976
{'loss': '3.398', 'grad_norm': '6.119', 'learning_rate': '4.379e-05', 'epoch': '0.9'}
[Validation Step 450] Epoch 0.9 | Validation Loss: 3.0377
{'eval_loss': '3.038', 'eval_runtime': '15.45', 'eval_samples_per_second': '129.5', 'eval_steps_per_second': '4.079', 'epoch': '0.9'}
[Training Step 500] Epoch 1.0 | Training Loss: 3.4486
{'loss': '3.449', 'grad_norm': '3.239', 'learning_rate': '4.171e-05', 'epoch': '1'}
[Validation Step 500] Epoch 1.0 | Validation Loss: 3.0305
{'eval_loss': '3.031', 'eval_runtime': '15.04', 'eval_samples_per_second': '132.9', 'eval_steps_per_second': '4.188', 'epoch': '1'}
```

```
[Training Step 550] Epoch 1.1 | Training Loss: 3.3524
{'loss': '3.352', 'grad_norm': '3.433', 'learning_rate': '3.963e-05', 'epoch': '1.1'}
[Validation Step 550] Epoch 1.1 | Validation Loss: 3.0387
{'eval_loss': '3.039', 'eval_runtime': '14.98', 'eval_samples_per_second': '133.5', 'eval_steps_per_second': '4.206', 'epoch': '1.1'}
[Training Step 600] Epoch 1.2 | Training Loss: 3.4488
{'loss': '3.449', 'grad_norm': '4.193', 'learning_rate': '3.754e-05', 'epoch': '1.2'}
[Validation Step 600] Epoch 1.2 | Validation Loss: 3.031
{'eval_loss': '3.031', 'eval_runtime': '14.99', 'eval_samples_per_second': '133.4', 'eval_steps_per_second': '4.202', 'epoch': '1.2'}
[Training Step 650] Epoch 1.3 | Training Loss: 3.3613
{'loss': '3.361', 'grad_norm': '3.822', 'learning_rate': '3.546e-05', 'epoch': '1.3'}
[Validation Step 650] Epoch 1.3 | Validation Loss: 3.0306
{'eval_loss': '3.031', 'eval_runtime': '15', 'eval_samples_per_second': '133.3', 'eval_steps_per_second': '4.199', 'epoch': '1.3'}
[Training Step 700] Epoch 1.4 | Training Loss: 3.3858
{'loss': '3.386', 'grad_norm': '3.688', 'learning_rate': '3.337e-05', 'epoch': '1.4'}
[Validation Step 700] Epoch 1.4 | Validation Loss: 3.0267
{'eval_loss': '3.027', 'eval_runtime': '15.01', 'eval_samples_per_second': '133.2', 'eval_steps_per_second': '4.197', 'epoch': '1.4'}
[Training Step 750] Epoch 1.5 | Training Loss: 3.3286
{'loss': '3.329', 'grad_norm': '2.997', 'learning_rate': '3.129e-05', 'epoch': '1.5'}
[Validation Step 750] Epoch 1.5 | Validation Loss: 3.0305
{'eval_loss': '3.03', 'eval_runtime': '15.01', 'eval_samples_per_second': '133.2', 'eval_steps_per_second': '4.197', 'epoch': '1.5'}
[Training Step 800] Epoch 1.6 | Training Loss: 3.3775
{'loss': '3.377', 'grad_norm': '3.564', 'learning_rate': '2.921e-05', 'epoch': '1.6'}
[Validation Step 800] Epoch 1.6 | Validation Loss: 3.0249
{'eval_loss': '3.025', 'eval_runtime': '15.39', 'eval_samples_per_second': '129.9', 'eval_steps_per_second': '4.093', 'epoch': '1.6'}
[Training Step 850] Epoch 1.7 | Training Loss: 3.3935
{'loss': '3.394', 'grad_norm': '3.879', 'learning_rate': '2.713e-05', 'epoch': '1.7'}
[Validation Step 850] Epoch 1.7 | Validation Loss: 3.0248
{'eval_loss': '3.025', 'eval_runtime': '14.99', 'eval_samples_per_second': '133.4', 'eval_steps_per_second': '4.201', 'epoch': '1.7'}
[Training Step 900] Epoch 1.8 | Training Loss: 3.3447
{'loss': '3.345', 'grad_norm': '5.303', 'learning_rate': '2.504e-05', 'epoch': '1.8'}
[Validation Step 900] Epoch 1.8 | Validation Loss: 3.0288
{'eval_loss': '3.029', 'eval_runtime': '14.99', 'eval_samples_per_second': '133.4', 'eval_steps_per_second': '4.201', 'epoch': '1.8'}
[Training Step 950] Epoch 1.9 | Training Loss: 3.4278
{'loss': '3.428', 'grad_norm': '3.582', 'learning_rate': '2.296e-05', 'epoch': '1.9'}
[Validation Step 950] Epoch 1.9 | Validation Loss: 3.0188
{'eval_loss': '3.019', 'eval_runtime': '15.03', 'eval_samples_per_second': '133.1', 'eval_steps_per_second': '4.192', 'epoch': '1.9'}
[Training Step 1000] Epoch 2.0 | Training Loss: 3.4391
{'loss': '3.439', 'grad_norm': '4.261', 'learning_rate': '2.087e-05', 'epoch': '2'}
```



```
[Validation Step 1000] Epoch 2.0 | Validation Loss: 3.0166
{'eval_loss': '3.017', 'eval_runtime': '15.03', 'eval_samples_per_second': '133.1',
 'eval_steps_per_second': '4.192', 'epoch': '2'}
[Training Step 1050] Epoch 2.1 | Training Loss: 3.3186
{'loss': '3.319', 'grad_norm': '3.874', 'learning_rate': '1.879e-05', 'epoch': '2.1'}
[Validation Step 1050] Epoch 2.1 | Validation Loss: 3.019
{'eval_loss': '3.019', 'eval_runtime': '15.04', 'eval_samples_per_second': '133', 'eval_steps_per_second': '4.19', 'epoch': '2.1'}
[Training Step 1100] Epoch 2.2 | Training Loss: 3.3443
{'loss': '3.344', 'grad_norm': '3.247', 'learning_rate': '1.671e-05', 'epoch': '2.2'}
[Validation Step 1100] Epoch 2.2 | Validation Loss: 3.017
{'eval_loss': '3.017', 'eval_runtime': '15.06', 'eval_samples_per_second': '132.8', 'eval_steps_per_second': '4.182', 'epoch': '2.2'}
[Training Step 1150] Epoch 2.3 | Training Loss: 3.3836
{'loss': '3.384', 'grad_norm': '3.167', 'learning_rate': '1.462e-05', 'epoch': '2.3'}
[Validation Step 1150] Epoch 2.3 | Validation Loss: 3.0158
{'eval_loss': '3.016', 'eval_runtime': '15.45', 'eval_samples_per_second': '129.5', 'eval_steps_per_second': '4.078', 'epoch': '2.3'}
[Training Step 1200] Epoch 2.4 | Training Loss: 3.3436
{'loss': '3.344', 'grad_norm': '2.883', 'learning_rate': '1.254e-05', 'epoch': '2.4'}
[Validation Step 1200] Epoch 2.4 | Validation Loss: 3.0153
{'eval_loss': '3.015', 'eval_runtime': '14.99', 'eval_samples_per_second': '133.4', 'eval_steps_per_second': '4.203', 'epoch': '2.4'}
[Training Step 1250] Epoch 2.5 | Training Loss: 3.2861
{'loss': '3.286', 'grad_norm': '4.484', 'learning_rate': '1.046e-05', 'epoch': '2.5'}
[Validation Step 1250] Epoch 2.5 | Validation Loss: 3.0183
{'eval_loss': '3.018', 'eval_runtime': '15.39', 'eval_samples_per_second': '130', 'eval_steps_per_second': '4.094', 'epoch': '2.5'}
[Training Step 1300] Epoch 2.6 | Training Loss: 3.3701
{'loss': '3.37', 'grad_norm': '3.539', 'learning_rate': '8.375e-06', 'epoch': '2.6'}
[Validation Step 1300] Epoch 2.6 | Validation Loss: 3.0179
{'eval_loss': '3.018', 'eval_runtime': '15.02', 'eval_samples_per_second': '133.1', 'eval_steps_per_second': '4.193', 'epoch': '2.6'}
[Training Step 1350] Epoch 2.7 | Training Loss: 3.3226
{'loss': '3.323', 'grad_norm': '3.664', 'learning_rate': '6.292e-06', 'epoch': '2.7'}
[Validation Step 1350] Epoch 2.7 | Validation Loss: 3.0177
{'eval_loss': '3.018', 'eval_runtime': '14.98', 'eval_samples_per_second': '133.5', 'eval_steps_per_second': '4.204', 'epoch': '2.7'}
[Training Step 1400] Epoch 2.8 | Training Loss: 3.384
{'loss': '3.384', 'grad_norm': '3.595', 'learning_rate': '4.208e-06', 'epoch': '2.8'}
[Validation Step 1400] Epoch 2.8 | Validation Loss: 3.0167
{'eval_loss': '3.017', 'eval_runtime': '15.08', 'eval_samples_per_second': '132.7', 'eval_steps_per_second': '4.179', 'epoch': '2.8'}
[Training Step 1450] Epoch 2.9 | Training Loss: 3.2978
{'loss': '3.298', 'grad_norm': '3.374', 'learning_rate': '2.125e-06', 'epoch': '2.9'}
[Validation Step 1450] Epoch 2.9 | Validation Loss: 3.0163
{'eval_loss': '3.016', 'eval_runtime': '14.98', 'eval_samples_per_second': '133.5', 'eval_steps_per_second': '4.205', 'epoch': '2.9'}
```

```

[Training Step 1500] Epoch 3.0 | Training Loss: 3.2887
{'loss': '3.289', 'grad_norm': '3.955', 'learning_rate': '4.167e-08', 'epoch': '3'}
[Validation Step 1500] Epoch 3.0 | Validation Loss: 3.0163
{'eval_loss': '3.016', 'eval_runtime': '15.37', 'eval_samples_per_second': '130.1',
'eval_steps_per_second': '4.098', 'epoch': '3'}
{'train_runtime': '1407', 'train_samples_per_second': '34.1', 'train_steps_per_second': '1.066', 'train_loss': '3.391', 'epoch': '3'}
[Timing] Fine-tuning duration: 23m 27s
[GPU 0] Memory - Allocated: 0.69 GB | Reserved: 5.68 GB
[GPU 1] Memory - Allocated: 0.02 GB | Reserved: 5.18 GB

[2.5/4] Evaluating fine-tuned model on held-out Test Set...
{'test_loss': '3.028', 'test_runtime': '14.95', 'test_samples_per_second': '133.8',
'test_steps_per_second': '4.215', 'epoch': '3'}
[Timing] Held-out test evaluation: 15.0s
[Test Set Metrics] Held-Out Test Loss: 3.0279
[Artifact] Held-out test evaluation metrics saved to 'test_results.json'

[3/4] Saving model weights and tokenizer...
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
[System Check] Model weights successfully saved to 'saved_model'

[4/4] Generating training artifacts...
[Artifact] Unified training/validation metrics log saved to 'training_log.csv'
[Artifact] Step-level loss graph exported to 'loss_curve.png'
=====
[Execution Summary] Device: CUDA GPU (Tesla T4) x2 | Samples: 20000 | Epochs: 3 | Total Duration: 23m 58s | Test Loss: 3.0279
-----
[Artifacts]
  Model      : /kaggle/working/saved_model
  Test Results: /kaggle/working/test_results.json
  Training Log: /kaggle/working/training_log.csv
  Loss Curve  : /kaggle/working/loss_curve.png
=====
[Complete] Fine-tuning pipeline executed successfully!

```

In [3]: `from datetime import datetime`

```

print("=" * 70)
print("Training Run Timestamp")
print("Date & Time:", datetime.now().strftime("%d-%m-%Y %H:%M:%S"))
print("=" * 70)

```

```

=====
Training Run Timestamp
Date & Time: 12-08-2026 04:32:50
=====

```

In [4]: `import zipfile`
`from pathlib import Path`

```

base_dir = Path("/kaggle/working")
zip_path = base_dir / "Group238_final_training_artifacts.zip"

with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:

```

```

# Add complete saved_model folder
model_dir = base_dir / "saved_model"

for file in model_dir.rglob("*"):
    if file.is_file():
        z.write(
            file,
            arcname=file.relative_to(base_dir)
        )

# Add generated artifacts
for filename in [
    "training_log.csv",
    "loss_curve.png",
    "test_results.json"
]:

    file = base_dir / filename

    if file.exists():
        z.write(
            file,
            arcname=filename
        )

print(f"Created: {zip_path}")
print(f"Size: {zip_path.stat().st_size / (1024**2):.2f} MB")

```

Created: /kaggle/working/Group238_final_training_artifacts.zip
 Size: 212.44 MB